weight-atlas Spec v2.1 — Robust Channel Scaling

> Status: Proposal | Target: patch release after v0.2.0 | Breaking: No (additive)

---

1. Problem Statement (v2.0)

The v2.0 spec applies `quantile_clip` **only** to the `rough` channel. The `height` and `tint` channels use `log1p` without outlier suppression. For models with heavy-tailed distributions (e.g. GGUF-quantized weights, MoE gating layers, or any model where a single layer's `spectral_norm` spikes), the Matplotlib sheet (which derives *everything* from `height`) collapses into a near-uniform field: one outlier dominates the colormap, pushing 98 % of values into the lowest 1–2 % of the color range.

Result: White or two-tone "spectrogram-like" images with no topographic readability. This is **not** a model-specific pathology; it is a scaling deficiency in the spec.

---

2. Design Principle

> **Comparable fingerprints require comparable *processes*, not comparable raw ranges.**

If every model is mapped through the *same* robust percentile-based pipeline, the resulting [0,1] fields are inter-comparable even though the per-model clip bounds differ. The algorithm is the constant; the bounds are data-derived.

---

3. Spec Changes

3.1 Channel Definitions (v2.1)

Channel	Statistic	v2.0 Transform	v2.1 Transform	Target Range
`height`	`spectral_norm`	`log1p`	`robust_scale(log1p(spectral_norm))`	[0,1]
`tint`	`stable_rank`	`log1p`	`robust_scale(log1p(stable_rank))`	[0,1]
`rough`	`kurtosis`	`quantile_clip(1–99%)` → [0,1]	`robust_scale(kurtosis)` *(unified)*	[0,1]

`robust_scale(x)` definition

```
def robust_scale(x, lower=0.01, upper=0.99):
    # 1. Compute q_lo = percentile(x, lower)
    #    Compute q_hi = percentile(x, upper)
    # 2. Clip x to [q_lo, q_hi]
    # 3. Min-max normalize clipped range to [0, 1]
    # 4. Record (q_lo, q_hi) in fingerprint.json for audit
    ...
```

Rationale for unifying `rough`: The current `quantile_clip` for `rough` is semantically identical to `robust_scale` with `lower=0.01, upper=0.99`. Unifying the naming simplifies the mental model: *every* channel goes through the same two-stage pipeline: **(1) compute statistic → (2) robust_scale**.

3.2 Fingerprint Schema Extension

`fingerprint.json` gains a `scaling` block at the top level:

```
{
  "spec_version": 2.1,
  "scaling": {
    "method": "robust_scale",
    "params": { "lower": 0.01, "upper": 0.99 },
    "channels": {
      "height": { "q_lo": 0.47, "q_hi": 3.89, "raw_min": 0.46, "raw_max": 5.56 },
      "tint":   { "q_lo": 0.12, "q_hi": 1.85, "raw_min": 0.12, "raw_max": 2.08 },
      "rough":  { "q_lo": -1.20, "q_hi": 4.50, "raw_min": -1.97, "raw_max": 2497.0 }
    }
  }
}
```

Fields:

- `q_lo`, `q_hi`: The actual clip bounds used. These are *not* secrets; they are audit metadata.
- `raw_min`, `raw_max`: The true min/max before clipping. Exposes outlier severity without distorting the image.
- `method` + `params`: Allow future spec versions to swap the algorithm (e.g. `asinh_scale`) while keeping fingerprints self-describing.

3.3 Degeneration Guard Updates

Current guards (`valid_fraction`, `normalized_std`) operate on the *raw* statistic. Add a **post-scaling guard**:

Guard	Condition	Action
`range_compression`	`(q_hi - q_lo) / (raw_max - raw_min) < 0.05`	CLI warning + UI banner: "Extreme outlier detected; 99 % of values compressed into <5 % of raw range."
`flat_field`	`normalized_std < 0.01` **after** scaling	Warning: "Field lacks variance post-scaling; sheet may be uninformative."

The `range_compression` guard is the *diagnostic* counterpart to the visual fix. It tells the user: "Yes, the image now looks good, but be aware that one monster outlier was clipped."

---

4. Pipeline Impact

4.1 TIFF Pipeline

No structural change. `rasterize` still produces `Field2D` with `NaN` for missing cells. The only difference is that the *values* written to `field_<ch>_raw.tif` are now in [0,1] instead of unbounded log-space.

Byte-identical determinism: `robust_scale` is deterministic (percentiles of a fixed array). The TIFF smoke test remains valid.

4.2 Matplotlib Sheet

The sheet becomes simpler, not more complex:

```
# BEFORE (v2.0)
height_raw = load_tiff("field_height_raw.tif")  # unbounded log1p values
q02, q98 = np.percentile(height_raw[~np.isnan(height_raw)], [2, 98])
contours = np.linspace(q02, q98, n_levels)
# Colormap applied to raw values; outlier destroys contrast

# AFTER (v2.1)
height_raw = load_tiff("field_height_raw.tif")  # already in [0,1]
contours = np.linspace(0.02, 0.98, n_levels)   # or spec.sheet.contour_levels
# Colormap applied to stable [0,1] field; contrast guaranteed
```

The "Contour Convention" section of ARCHITECTURE.md should be updated:

> ~~Contours on the 2D sheet use deterministic, comparable levels: `np.linspace(q02, q98, ...)` computed from the raw height field (before normalization).~~
> Contours on the 2D sheet use deterministic, comparable levels: `np.linspace(0.02, 0.98, spec.sheet.contour_levels)` applied to the **scaled height field** (already in [0,1]). Because `robust_scale` guarantees a well-distributed [0,1] range, fixed percentile levels are globally comparable without per-model recomputation.

4.3 Blender Pipeline

No change required. Blender reads `field_height_smooth.tif` and `field_tint_smooth.tif`. If these are now [0,1]-scaled, the existing `z_scale` and vertex-color logic works unchanged (it already assumes normalized inputs).

4.4 JSON Manifest

`manifest.json` already lists `field_height_raw.tif`, etc. No new artefacts. The `fingerprint.json` schema extension is the only JSON change.

---

5. Compare Layer Impact

5.1 Delta Computation

The compare pipeline (`delta.py`) currently computes Δ on "scaled channel values (B - A after applying channel scale)". Under v2.1, "scaled" means [0,1] for all channels. This is *more* robust than v2.0 because:

- In v2.0, `height` delta is computed on unbounded log1p values. A single outlier in A or B creates a massive delta spike that is 99 % artifact, 1 % signal.
- In v2.1, both A and B are clipped to their respective 1–99 % ranges and mapped to [0,1]. The delta reflects *relative* positional shifts within the robust range, not absolute outlier magnitude.

5.2 Diverging Colormap Limits

The `diverging_clip` parameter (default 0.98) in `compare` becomes more meaningful:

```
# v2.0: symmetric limits on unbounded height delta → often dominated by one outlier
lim = np.quantile(np.abs(delta_height), 0.98)

# v2.1: symmetric limits on [0,1] delta → naturally bounded, outliers already suppressed
lim = np.quantile(np.abs(delta_height), 0.98)  # same code, better behavior
```

5.3 Backwards Compatibility

Scenario	Behavior
Compare v2.1 ↔ v2.1	Full feature set, strict mode works as designed
Compare v2.0 ↔ v2.1	**Hard-reject** (spec_version mismatch). This is existing policy and correct: the meaning of `height` values differs (unbounded log vs [0,1]).
Compare v2.1 ↔ v2.1 (different `robust_scale` params)	Allowed. The `scaling` block in fingerprint.json documents the parameters. A future spec could add "scaling param mismatch → warning".

---

6. Migration Path

6.1 For Existing v2.0 Artefacts

No automatic migration. v2.0 fingerprints must be re-scanned. The existing `spec_version` gate already enforces this.

6.2 Code Changes (checklist)

- `weight_atlas/stats/compute.py`: Apply `robust_scale` after `log1p` for `height` and `tint`; rename `rough` path to use same function.
- `weight_atlas/fields/degenerations.py`: Add `range_compression` guard.
- `weight_atlas/render/sheet.py`: Simplify contour level computation; remove per-model percentile logic for `height`.
- `weight_atlas/fingerprint.py`: Include `scaling` block in JSON output.
- `atlas_spec.v2.json` -> `atlas_spec.v2.1.json`: Update channel definitions.
- `tests/`: Update fixture fingerprints (re-scan Bonsai-8B). Add unit test for `range_compression` guard.
- `ARCHITECTURE.md`: Update "Contour Convention" and "Channel" sections.

6.3 CLI / UX

```
# Re-scan with v2.1 (automatic if tool is updated)
weight-atlas scan ./models/bonsai-8b.gguf --out ./artefacts_v2_1

# Expect: no "mostly white" sheet; expect: range_compression warning if outliers exist
weight-atlas diagnose ./artefacts_v2_1/fingerprint.json
```

```
# -> "range_compression: mild (q_range covers 12 % of raw_range)"
```

---

7. Open Questions (for team discussion)

1. **Should `lower/upper` be configurable per-channel in `atlas_spec`?**
   Pro: Flexibility for future research. *Con:* Configuration drift undermines comparability. **Recommendation:** Hard-code `1–99 %` in v2.1; make configurable only in a future major version if empirical evidence demands it.

2. **Should `raw_min/raw_max` include *unclipped* extremes or *pre-log1p* extremes?**
   Proposed: Store both stages: `raw_stat_min/max` (before log1p) and `raw_scale_min/max` (after log1p, before clip). This gives full auditability.

3. **GGUF dequantization noise:**
   If Q4_0 quantization itself introduces the 259 outlier in `spectral_norm`, `robust_scale` will suppress it visually but the `range_compression` guard will flag it. Is this desirable? **Yes:** The fingerprint remains honest about data quality while remaining readable.

---

8. Summary

Concern	v2.0 State	v2.1 Fix
White sheets	`height` unbounded -> outlier saturates colormap	`robust_scale` on all channels -> guaranteed contrast
Compare validity	Delta on unbounded `height` -> artifact spikes	Delta on [0,1] fields -> stable divergence maps
Auditability	No record of clip bounds	`scaling` block in fingerprint.json
Degeneration detection	`valid_fraction`, `normalized_std` only	+ `range_compression` guard
Architecture churn	—	Minimal: one function (`robust_scale`) applied universally; sheet renderer simplified

Verdict: The architecture is sound. The spec needs this one correction to make the visual fingerprint mission viable.